

```

class TreeNode {
    Object elem;
    TreeNode left;
    TreeNode right;

    TreeNode(Object elem) {
        this.elem = elem;
        this.left = null;
        this.right = null;
    }
}

```

```

public class BinaryTreeTasks {

    // 1.1 Pre-order
    public static void preOrder(TreeNode root) {

        if (root == null) {
            return;
        }

        System.out.println(root.elem);

        preOrder(root.left);
        preOrder(root.right);
    }

    // 1.2 Post-order + level
    public static void postOrderLevels(TreeNode root, int level) {

        if (root == null) {
            return;
        }

        postOrderLevels(root.left, level + 1);
        postOrderLevels(root.right, level + 1);

        System.out.println(root.elem + ":" + level);
    }

    // 1.3 In-order + even levels

```

```

public static void inOrderEvenLevels(TreeNode root, int level) {

    if (root == null) {
        return;
    }

    inOrderEvenLevels(root.left, level + 1);

    if (level % 2 == 0) {
        System.out.println(root.elem + ":" + level);
    }

    inOrderEvenLevels(root.right, level + 1);
}

```

// 2. Count nodes

```

public static int countNodes(TreeNode root) {

    if (root == null) {
        return 0;
    }

    return 1
        + countNodes(root.left)
        + countNodes(root.right);
}

```

// 3.1 Sum

```

public static int sumNodes(TreeNode root) {

    if (root == null) {
        return 0;
    }

    return (Integer) root.elem
        + sumNodes(root.left)
        + sumNodes(root.right);
}

```

// 3.2 Print sum

```

public static void printSum(TreeNode root) {

```

```
    System.out.println(sumNodes(root));  
}
```

// 4. Print leaves

```
public static void printLeaves(TreeNode root) {  
  
    if (root == null) {  
        return;  
    }  
  
    if (root.left == null && root.right == null) {  
        System.out.println(root.elem);  
        return;  
    }  
  
    printLeaves(root.left);  
    printLeaves(root.right);  
}
```

// 5. Maximum

```
public static int findMax(TreeNode root) {  
  
    if (root == null) {  
        return Integer.MIN_VALUE;  
    }  
  
    int leftMax = findMax(root.left);  
    int rightMax = findMax(root.right);  
  
    int current = (Integer) root.elem;  
  
    int max = current;  
  
    if (leftMax > max) {  
        max = leftMax;  
    }  
  
    if (rightMax > max) {  
        max = rightMax;  
    }  
}
```

```
    return max;
}
```

```
// 6. Subtraction of nodes
```

```
public static int subtract_sum(TreeNode root) {

    if (root == null) {
        return 0;
    }

    return sumNodes(root.left) - sumNodes(root.right);
}
```

```
// 7. Difference of level sum
```

```
public static int levelDifference(TreeNode root) {

    return levelDifferenceHelper(root, 0);
}

public static int levelDifferenceHelper(TreeNode root, int level) {

    if (root == null) {
        return 0;
    }

    int value = (Integer) root.elem;

    if (level % 2 == 0) {
        value = -value;
    }

    return value
        + levelDifferenceHelper(root.left, level + 1)
        + levelDifferenceHelper(root.right, level + 1);
}
```

```
// 8. Swap children at level M and above
```

```
public static void swap_child(TreeNode root, int level, int M) {

    if (root == null) {
        return;
    }
}
```

```
}  
  
if (level >= M) {  
  
    TreeNode temp = root.left;  
    root.left = root.right;  
    root.right = temp;  
}  
  
swap_child(root.left, level + 1, M);  
swap_child(root.right, level + 1, M);  
}  
}
```